

SIMATS ENGINEERING
SAVEETHA SCHOOL OF ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL – 3
CODING CHALLENGE

Course Code	DSA0135	Course Title	Object Oriented Programming for Real Time Applications
CO Assessed	CO2 (BL3)	Assessment	Tool – 3
Weightage	10%	Total Marks	100
Student Name		Reg No	
Section / Batch		Date	
CO Statement	Design C++ programs by applying functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to solve real-world programming problems. (BL3)		

RUBRICS FOR EVALUATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. A university aims to develop a Student Analytics System to evaluate academic performance across multiple departments, where each department has a varying number of students and their marks are stored in a two-dimensional structure. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage student data effectively.
 - A. The program should compute the topper in each department, identify the overall university topper, and calculate the pass percentage for each department.
 - B. Additionally, the solution must handle variable department sizes, edge cases such as empty departments, equal marks, or all students failing, and ensure efficient comparison across departments while maintaining optimized time complexity and clean modular design.

Program

```
#include <iostream>
using namespace std;
class Department{
private:
    int deptId;
    int n;
    int marks[100];
public:
    static int totalDepartments;
    void input(){
        cout << "\nEnter Department ID: ";
        cin >> deptId;
        cout << "Enter Number of Students: ";
        cin >> n;
        if (n > 100)
            n = 100;
        if (n > 0){
            cout << "Enter Marks:\n";
            for (int i = 0; i < n; i++){
                cin >> marks[i];
            }
        }
    }
    int getTopperMark(){
        if (n == 0)
            return -1;
        int top = marks[0];
        for (int i = 1; i < n; i++){
            if (marks[i] > top)
                top = marks[i];
        }
        return top;
    }
    double getPassPercentage(){
        if (n == 0)
```

```

        return 0;
    int pass = 0;
    for (int i = 0; i < n; i++){
        if (marks[i] >= 40)
            pass++;
    }
    return (pass * 100.0) / n;
}
void displayDepartmentResult(){
    cout << "\nDepartment " << deptId;
    if (n == 0){
        cout << "\nNo Students in Department\n";
        return;
    }
    cout << "\nTopper Mark = " << getTopperMark();
    cout << "\nPass Percentage = " << getPassPercentage() << "%\n";
}
};
int Department::totalDepartments = 0;
int main(){
    int d;
    cout << "Enter Number of Departments: ";
    cin >> d;
    Department dept[50]; // Array of Objects
    int overallTopper = -1;
    int overallDept = -1;
    for (int i = 0; i < d; i++){
        dept[i].input();
        Department::totalDepartments++;
        int deptTop = dept[i].getTopperMark();
        if (deptTop > overallTopper){
            overallTopper = deptTop;
            overallDept = i + 1;
        }
    }
    cout << "\n===== DEPARTMENT RESULTS =====\n";
    for (int i = 0; i < d; i++){
        dept[i].displayDepartmentResult();
    }
    cout << "\n===== UNIVERSITY TOPPER =====\n";
    if (overallTopper == -1){
        cout << "No Students in University\n";
    }
    else{
        cout << "Overall Topper Mark = "
            << overallTopper
            << "\nDepartment Number = "
            << overallDept << endl;
    }
    cout << "\nTotal Departments Processed = "

```

```
    << Department::totalDepartments << endl;  
    return 0;  
}
```

TEST CASE 1 (Normal Case)

Input:

```
2  
101  
3  
75 80 90  
102  
4  
60 45 35 85
```

Output:

```
Department 101  
Topper Mark = 90  
Pass Percentage = 100%
```

```
Department 102  
Topper Mark = 85  
Pass Percentage = 75%
```

```
Overall Topper Mark = 90  
Department Number = 1
```

TEST CASE 2 (Empty Department)

Input:

```
1  
101  
0
```

Output:

```
No Students in Department  
No Students in University
```

TEST CASE 3 (All Students Fail)

Input:

```
1  
101  
5  
10 20 30 35 39
```

Output:

Topper Mark = 39
Pass Percentage = 0%

TEST CASE 4 (Equal Top Marks)

Input:
1
101
4
95 95 80 70

Output:
Topper Mark = 95
Pass Percentage = 100%

TEST CASE 5 (Single Student)

Input:
1
101
1
100

Output:
Topper Mark = 100
Pass Percentage = 100%
Overall Topper Mark = 100

TIME COMPLEXITY

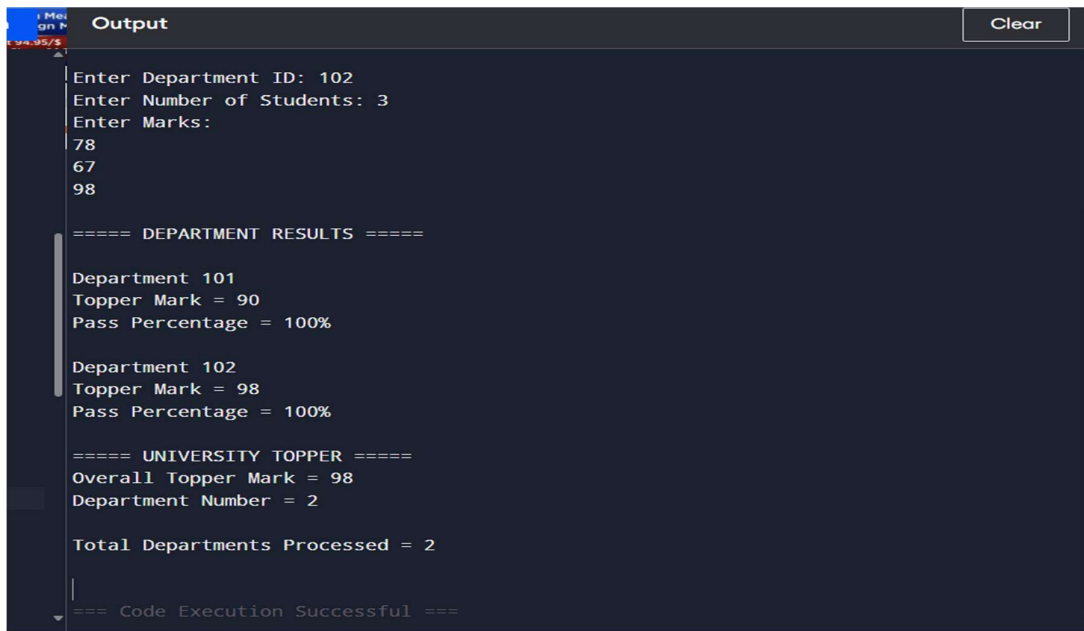
Best Case:
 $O(1)$
(One department with one student)

Average Case:
 $O(N)$
(N = Total number of students)

Worst Case:
 $O(N)$
(All student marks must be traversed)

SPACE COMPLEXITY

$O(N)$
(for storing student marks)



```
Output
Enter Department ID: 102
Enter Number of Students: 3
Enter Marks:
78
67
98

===== DEPARTMENT RESULTS =====

Department 101
Topper Mark = 90
Pass Percentage = 100%

Department 102
Topper Mark = 98
Pass Percentage = 100%

===== UNIVERSITY TOPPER =====
Overall Topper Mark = 98
Department Number = 2

Total Departments Processed = 2

=== Code Execution Successful ===
```

2. A country conducts elections across multiple regions, where votes for different candidates are stored in a two-dimensional structure representing regions and candidates. Develop a C++ program using object-oriented principles and functions to store and process vote data efficiently.
 - A. The program should determine the winner in each region, compute the overall winner by aggregating votes across all regions, and detect tie conditions at both regional and overall levels.
 - B. The implementation should use appropriate access specifiers, member functions, and static members where necessary, and must handle large input sizes efficiently. Additionally, the program should address edge cases such as ties, zero votes, and invalid inputs, while ensuring accurate multi-level comparisons and optimized performance.

Program

```
#include<iostream>
using namespace std;
class Election{
private:
    int regions,candidates;
    int votes[50][100];
public:
    static int totalRegions;
    void input(){
        cout<<"Enter Number of Regions: ";
        cin>>regions;
        cout<<"Enter Number of Candidates: ";
        cin>>candidates;
        if(regions>50)
            regions=50;
        if(candidates>100)
            candidates=100;
        for(int i=0;i<regions;i++){
```

```

        cout<<"\nEnter Votes for Region "<<i+1<<":\n";
        for(int j=0;j<candidates;j++){
            cin>>votes[i][j];
            if(votes[i][j]<0)
                votes[i][j]=0;
        }
    }
    totalRegions=regions;
}

void regionalWinners(){
    cout<<"\n===== REGIONAL RESULTS =====\n";
    for(int i=0;i<regions;i++){
        int maxVotes=votes[i][0];
        int winner=0,tie=0;
        for(int j=1;j<candidates;j++){
            if(votes[i][j]>maxVotes){
                maxVotes=votes[i][j];
                winner=j;
                tie=0;
            }
            else if(votes[i][j]==maxVotes){
                tie=1;
            }
        }
        cout<<"Region "<<i+1<<" : ";
        if(tie)
            cout<<"Tie";
        else
            cout<<"Winner = Candidate "<<winner+1;
        cout<<endl;
    }
}

void overallWinner(){
    int totalVotes[100]={0};
    for(int i=0;i<regions;i++){
        for(int j=0;j<candidates;j++){
            totalVotes[j]+=votes[i][j];
        }
    }
    int maxVotes=totalVotes[0];
    int winner=0,tie=0;
    for(int j=1;j<candidates;j++){
        if(totalVotes[j]>maxVotes){
            maxVotes=totalVotes[j];
            winner=j;
            tie=0;
        }
        else if(totalVotes[j]==maxVotes){
            tie=1;
        }
    }
}

```

```

    }
    cout<<"\n===== OVERALL RESULT =====\n";
    if(tie)
        cout<<"Overall Election Result : Tie";
    else
        cout<<"Overall Winner = Candidate "<<winner+1
            <<" with "<<maxVotes<<" votes";
    cout<<"\nTotal Regions Processed = "
        <<Election::totalRegions<<endl;
    }
};
int Election::totalRegions=0;
int main(){
    Election e;
    e.input();
    e.regionalWinners();
    e.overallWinner();
    return 0;
}

```

TEST CASE 1 (Normal Case)

Input:

```

3
3
100 80 60
90 120 70
110 100 50

```

Output:

```

Region 1 : Winner = Candidate 1
Region 2 : Winner = Candidate 2
Region 3 : Winner = Candidate 1

```

Overall Winner = Candidate 1 with 300 votes

TEST CASE 2 (Regional Tie)

Input:

```

1
3
100 100 50

```

Output:

```

Region 1 : Tie
Overall Election Result : Tie

```

TEST CASE 3 (Overall Tie)

Input:

2
2
100 50
50 100

Output:

Region 1 : Winner = Candidate 1
Region 2 : Winner = Candidate 2
Overall Election Result : Tie

TEST CASE 4 (All Zero Votes)

Input:

2
3
0 0 0
0 0 0

Output:

Region 1 : Tie
Region 2 : Tie
Overall Election Result : Tie

TEST CASE 5 (Invalid Votes)

Input:

1
3
100 -50 200

Output:

Region 1 : Winner = Candidate 3
Overall Winner = Candidate 3 with 200 votes

(-50 is converted to 0)

TIME COMPLEXITY

Best Case:

$O(R \times C)$
(Need to scan all votes at least once)

Average Case:

$O(R \times C)$

Worst Case:

$O(R \times C)$

Where:

R = Number of Regions

C = Number of Candidates

SPACE COMPLEXITY

$O(R \times C)$

For storing the 2D votes array.

```
Output
Enter Number of Regions: 2
Enter Number of Candidates: 2
Enter Votes for Region 1:
0
12
Enter Votes for Region 2:
12
0
===== REGIONAL RESULTS =====
Region 1 : Winner = Candidate 2
Region 2 : Winner = Candidate 1
===== OVERALL RESULT =====
Overall Election Result : Tie
Total Regions Processed = 2

=== Code Execution Successful ===
```

-
3. In this scenario, a Retail Chain Management System is designed to analyze product sales across multiple stores using object-oriented programming in C++.
 - A. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage product data effectively. Each store contains a collection of products, and every product has attributes such as product name, price, and quantity sold and compute individual product sales ($\text{price} \times \text{quantity}$), and display the information. An array of objects is used to store multiple products for each store, allowing the program to scale efficiently for different store sizes.
 - B. To manage multiple stores, the program can either use a two-dimensional array of objects or create another class such as Store, which contains an array of Product objects. For each store, the total sales are calculated by iterating through all its products and summing their individual sales using a dedicated member function like `calculateStoreSales()`. A static data member is used to maintain the overall

total sales across all stores, demonstrating the use of shared class-level data. This helps in tracking cumulative performance of the entire retail chain.

- C. The program also includes logic to identify the best-performing store, which is the store with the highest total sales, and the best-selling product, which is the product generating the highest revenue across all stores.
- D. Proper input/output handling is essential, where the user is prompted to enter the number of stores, number of products per store, and corresponding product details. The output should be well-structured, displaying store-wise sales, overall sales, best-performing store, and top product.

Program

```
#include<iostream>
#include<string>
using namespace std;
class Product{
private:
    string pname;
    float price;
    int qty;

public:
    void input(){
        cout<<"Enter Product Name: ";
        cin>>pname;
        cout<<"Enter Price: ";
        cin>>price;
        cout<<"Enter Quantity Sold: ";
        cin>>qty;
    }
    float calculateSales(){
        return price*qty;
    }
    string getName(){
        return pname;
    }
    void display(){
        cout<<"\nProduct Name : "<<pname;
        cout<<"\nPrice : "<<price;
        cout<<"\nQuantity Sold : "<<qty;
        cout<<"\nSales : "<<calculateSales()<<endl;
    }
};
class Store{
private:
    int storeId;
    int n;
    Product p[100];
public:
```

```

static float overallSales;
void input(){
    cout<<"\nEnter Store ID: ";
    cin>>storeId;
    cout<<"Enter Number of Products: ";
    cin>>n;
    if(n>100)
        n=100;
    for(int i=0;i<n;i++){
        cout<<"\nProduct "<<i+1<<endl;
        p[i].input();
    }
}
float calculateStoreSales(){
    float total=0;
    for(int i=0;i<n;i++)
        total+=p[i].calculateSales();
    overallSales+=total;
    return total;
}
void displayStoreDetails(){
    cout<<"\n===== Store "<<storeId<<" =====\n";
    for(int i=0;i<n;i++)
        p[i].display();
    cout<<"Store Total Sales : "<<calculateStoreSales()<<endl;
}
float getStoreSales(){
    float total=0;
    for(int i=0;i<n;i++)
        total+=p[i].calculateSales();
    return total;
}
int getProductCount(){
    return n;
}
Product getProduct(int index){
    return p[index];
}
int getStoreId(){
    return storeId;
}
};
float Store::overallSales=0;
int main(){
    int stores;
    cout<<"Enter Number of Stores: ";
    cin>>stores;
    Store s[50];
    for(int i=0;i<stores;i++)
        s[i].input();
}

```

```

int bestStore=0;
float maxStoreSales=s[0].getStoreSales();
string bestProduct;
float maxProductSales=-1;
cout<<"\n===== STORE DETAILS =====\n";
for(int i=0;i<stores;i++){
    s[i].displayStoreDetails();
    float storeSales=s[i].getStoreSales();
    if(storeSales>maxStoreSales){
        maxStoreSales=storeSales;
        bestStore=i;
    }
    for(int j=0;j<s[i].getProductCount();j++){
        Product temp=s[i].getProduct(j);
        if(temp.calculateSales()>maxProductSales){
            maxProductSales=temp.calculateSales();
            bestProduct=temp.getName();
        }
    }
}
cout<<"\n===== RETAIL CHAIN REPORT =====\n";
cout<<"Overall Sales : "<<Store::overallSales<<endl;
cout<<"Best Performing Store : Store "<<s[bestStore].getStoreId()<<endl;
cout<<"Best Selling Product : "<<bestProduct<<endl;
cout<<"Highest Product Revenue : "<<maxProductSales<<endl;
return 0;
}

```

TEST CASE 1 (Normal Case)

Input:

2

101

2

Pen

10

20

Book

50

10

102

2

Pencil

5

50

Bag

500

5

Output:

Store 101 Total Sales = 700

Store 102 Total Sales = 2750

Overall Sales = 3450

Best Performing Store = 102

Best Selling Product = Bag

Highest Product Revenue = 2500

TEST CASE 2 (Single Store, Single Product)

Input:

1

101

1

Pen

10

5

Output:

Store 101 Total Sales = 50

Overall Sales = 50

Best Performing Store = 101

Best Selling Product = Pen

Highest Product Revenue = 50

TEST CASE 3 (Store with No Products)

Input:

1

101

0

Output:

Store 101 Total Sales = 0

Overall Sales = 0

Best Performing Store = 101

TEST CASE 4 (Product with Zero Quantity)

Input:

1

101

2

Pen

10

0

Book

50

5

Output:

Pen Sales = 0

Book Sales = 250

Store Total Sales = 250

Overall Sales = 250

Best Selling Product = Book

TEST CASE 5 (Equal Revenue Products)

Input:

1

101

2

Pen

100

10

Book

50

20

Output:

Pen Sales = 1000

Book Sales = 1000

Store Total Sales = 2000

Best Selling Product = Pen

(First encountered product is selected)

TEST CASE 6 (All Sales Zero)

Input:

1

101

3
A
10
0
B
20
0
C
30
0

Output:
Overall Sales = 0
Highest Product Revenue = 0

TIME COMPLEXITY

Best Case:
 $O(S \times P)$
(One store with one product)

Average Case:
 $O(S \times P)$

Worst Case:
 $O(S \times P)$

Where:
S = Number of Stores
P = Number of Products per Store

```
Output
===== Store 2 =====
Product Name : i3
Price : 45
Quantity Sold : 300
Sales : 13500

Product Name : i4
Price : 39
Quantity Sold : 89
Sales : 3471

Product Name : i5
Price : 45
Quantity Sold : 3
Sales : 135
Store Total Sales : 17106

===== RETAIL CHAIN REPORT =====
Overall Sales : 28256
Best Performing Store : Store 2
Best Selling Product : i3
Highest Product Revenue : 13500
```